02-01 Validator 구조체 및 초기화 과정 분석

개요

Solana의 `Validator` 는 전체 블록체인 노드의 핵심 컴포넌트로, 모든 서비스를 조율하고 관리하는 중앙 집중식 구조체입니다. 이 문서에서는 `core/src/validator.rs` 에 정의된 `Validator` 구조체와 그 초기화 과정인 `Validator::new()` 함수를 상세히 분석합니다.

Validator 구조체 정의

핵심 구조체

```
pub struct Validator {
validator_exit: Arc<RwLock<Exit>>,
json_rpc_service: Option<JsonRpcService>,
pubsub_service: Option<PubSubService>,
rpc_completed_slots_service: JoinHandle<()>,
optimistically_confirmed_bank_tracker: Option<OptimisticallyConfirmedBankTracker
transaction_status_service: Option<TransactionStatusService>,
rewards_recorder_service: Option<RewardsRecorderService>,
cache_block_meta_service: Option<CacheBlockMetaService>,
entry_notifier_service: Option<EntryNotifierService>,
system_monitor_service: Option<SystemMonitorService>,
sample_performance_service: Option<SamplePerformanceService>,
```

```
poh_timing_report_service: PohTimingReportService,

stats_reporter_service: StatsReporterService,

gossip_service: GossipService,

serve_repair_service: ServeRepairService,

completed_data_sets_service: CompletedDataSetsService,

snapshot_packager_service: Option<SnapshotPackagerService>,

poh_recorder: Arc<RwLock<PohRecorder>>,

poh_service: PohService,

tpu: Tpu,

tvu: Tvu,

ip_echo_server: Option<solana_net_utils::IpEchoServer>,

pub  cluster_info: Arc<ClusterInfo>,

pub  bank_forks: Arc<RwLock<BankForks>>,

pub  blockstore: Arc<Blockstore>,

geyser_plugin_service: Option<GeyserPluginService>,

blockstore_metric_report_service: BlockstoreMetricReportService,

accounts_background_service: AccountsBackgroundService,

accounts_hash_verifier: AccountsHashVerifier,

turbine_quic_endpoint: Option<Endpoint>,

turbine_quic_endpoint_runtime: Option<TokioRuntime>,

turbine_quic_endpoint_join_handle: Option<solana_turbine::quic_endpoint::AsyncTr

repair_quic_endpoint: Option<Endpoint>,

repair_quic_endpoint_runtime: Option<TokioRuntime>,
```

```
repair_quic_endpoint_join_handle: Option<repair::quic_endpoint::AsyncTryJoinHand

}
```

구조체 필드 분류

1. 핵심 처리 유닛

- **`tpu: Tpu`** : Transaction Processing Unit - 트랜잭션 처리 파이프라인
- **`tvu: Tvu`** : Transaction Validation Unit - 트랜잭션 검증 파이프라인
- **`poh_service: PohService`** : Proof of History 서비스
- **`poh_recorder: Arc<RwLock<PohRecorder>>`** : PoH 기록기

2. 데이터 저장 및 관리

- **`bank_forks: Arc<RwLock<BankForks>>`** : 블록체인 상태 관리
- **`blockstore: Arc<Blockstore>`** : 블록 데이터 저장소
- **`cluster_info: Arc<ClusterInfo>`** : 클러스터 정보 관리

3. 네트워크 서비스

- **`gossip_service: GossipService`** : 가십 프로토콜 서비스
- **`serve_repair_service: ServeRepairService`** : 복구 서비스
- **`json_rpc_service: Option<JsonRpcService>`** : JSON-RPC API 서비스
- **`pubsub_service: Option<PubSubService>`** : 발행-구독 서비스

4. 백그라운드 서비스

- **`accounts_background_service: AccountsBackgroundService`** : 계정 백그라운드 처리
- **`accounts_hash_verifier: AccountsHashVerifier`** : 계정 해시 검증

- **`system_monitor_service: Option<SystemMonitorService>`** : 시스템 모니터링
- **`stats_reporter_service: StatsReporterService`** : 통계 보고 서비스

Validator::new() 함수 분석

함수 시그니처

```
pub fn new(
mut node: Node,
identity_keypair: Arc<Keypair>,
ledger_path: &Path,
vote_account: &Pubkey,
authorized_voter_keypairs: Arc<RwLock<Vec<Arc<Keypair>>>>,
cluster_entrypoints: Vec<ContactInfo>,
config: &ValidatorConfig,
should_check_duplicate_instance: bool,
rpc_to_plugin_manager_receiver: Option<Receiver<GeyserPluginManagerRequest>>,
start_progress: Arc<RwLock<ValidatorStartProgress>>,
socket_addr_space: SocketAddrSpace,
use_quic: bool,
tpu_connection_pool_size: usize,
tpu_enable_udp: bool,
admin_rpc_service_post_init: Arc<RwLock<Option<AdminRpcRequestMetadataPostInit>>
```

```
) -> Result<Self, String>
```

초기화 과정 단계별 분석

1단계: 기본 설정 및 검증 (라인 ~550-600)

```
let id = identity_keypair.pubkey();
assert_eq!(&id, node.info.pubkey());

info!("identity pubkey: {id}");
info!("vote account pubkey: {vote_account}");

if !config.no_os_network_stats_reporting {
verify_net_stats_access()
.map_err(|err| format!("Failed to access network stats: {err:?}"))?;
}
```

주요 작업:

- 노드 신원 확인 및 로깅
- 네트워크 통계 접근 권한 검증
- 기본 exit 신호 설정

2단계: Geyser 플러그인 초기화 (라인 ~600-650)

```
let geyser_plugin_service =
if let Some(geyser_plugin_config_files) = &config.on_start_geyser_plugin_config
```

```
let (confirmed_bank_sender, confirmed_bank_receiver) = unbounded();
bank_notification_senders.push(confirmed_bank_sender);
Some(
GeyserPluginService::new_with_receiver(
confirmed_bank_receiver,
geyser_plugin_config_files,
rpc_to_plugin_manager_receiver_and_exit,
)
.map_err(|err| format!("Failed to load the Geyser plugin: {err:?}"))?,
)
} else {
None
};
```

주요 작업:

- Geyser 플러그인 서비스 초기화
- 은행 알림 채널 설정
- 플러그인 매니저와의 통신 채널 구성

3단계: 투표 설정 및 클러스터 정보 (라인 ~650-700)

```
if  config.voting_disabled {
warn!("voting disabled");
authorized_voter_keypairs.write().unwrap().clear();
```

```
} else {
for  authorized_voter_keypair  in  authorized_voter_keypairs.read().unwrap().ite
warn!("authorized voter: {}", authorized_voter_keypair.pubkey());
}
}

for  cluster_entrypoint  in &cluster_entrypoints {
info!("entrypoint: {:?}", cluster_entrypoint);
}
```

주요 작업:

- 투표 기능 활성화/비활성화 설정
- 승인된 투표자 키페어 관리
- 클러스터 진입점 정보 로깅

4단계: 시스템 초기화 (라인 ~700-750)

```
if rayon::ThreadPoolBuilder::new()
.thread_name(|i| format!("solRayonGlob{i:02}"))
.build_global()
.is_err()
{
warn!("Rayon global thread pool already initialized");
}
```

```
if solana_perf::perf_libs::api().is_some() {
info!("Initializing sigverify, this could take a while...");
} else {
info!("Initializing sigverify...");
}
sigverify::init();
info!("Initializing sigverify done.");
```

주요 작업:

- Rayon 글로벌 스레드 풀 초기화
- 서명 검증 시스템 초기화
- 성능 라이브러리 확인

5단계: 제네시스 설정 및 블록스토어 정리 (라인 ~750-850)

```
let genesis_config =
open_genesis_config(ledger_path, config.max_genesis_archive_unpacked_size)
.map_err(|err| format!("Failed to open genesis config: {err}"))?;

metrics_config_sanity_check(genesis_config.cluster_type)?;

if let Some(expected_shred_version) = config.expected_shred_version {
if let Some(wait_for_supermajority_slot) = config.wait_for_supermajority {
*start_progress.write().unwrap() = ValidatorStartProgress::CleaningBlockStore;
```

```
backup_and_clear_blockstore(
ledger_path,
config,
wait_for_supermajority_slot + 1,
expected_shred_version,
)?;
}
}
```

주요 작업:

- 제네시스 설정 로드 및 검증
- 메트릭 설정 검사
- 필요시 블록스토어 백업 및 정리

6단계: 계정 경로 정리 (라인 ~850-900)

```
info!("Cleaning accounts paths..");
*start_progress.write().unwrap() = ValidatorStartProgress::CleaningAccounts;
let mut timer = Measure::start("clean_accounts_paths");
cleanup_accounts_paths(config);
timer.stop();
info!("Cleaning accounts paths done. {timer}");
```

```
snapshot_utils::purge_incomplete_bank_snapshots(&config.snapshot_config.bank_sna
snapshot_utils::purge_old_bank_snapshots_at_startup(
```

```
&config.snapshot_config.bank_snapshots_dir,
);
```

주요 작업:

- 계정 저장 경로 정리
- 불완전한 스냅샷 제거
- 오래된 스냅샷 정리

7단계: 블록스토어 로드 및 처리 (라인 ~950-1000)

```
let (
genesis_config,
bank_forks,
blockstore,
original_blockstore_root,
ledger_signal_receiver,
completed_slots_receiver,
leader_schedule_cache,
starting_snapshot_hashes,
TransactionHistoryServices { ... },
blockstore_process_options,
blockstore_root_scan,
pruned_banks_receiver,
entry_notifier_service,
```

```
) = load_blockstore(
config,
ledger_path,
exit.clone(),
&start_progress,
accounts_update_notifier,
transaction_notifier,
entry_notifier,
Some(poh_timing_point_sender.clone()),
)?;
```

주요 작업:

- 블록스토어 로드 및 초기화
- 은행 포크 설정
- 리더 스케줄 캐시 생성
- 트랜잭션 히스토리 서비스 설정

8단계: 네트워크 정보 설정 (라인 ~1000-1050)

```
node.info.set_wallclock(timestamp());
node.info.set_shred_version(compute_shred_version(
&genesis_config.hash(),
Some(&hard_forks),
));
```

```
Self::print_node_info(&node);
```

주요 작업:

- 노드 정보 업데이트 (타임스탬프, shred 버전)
- 하드 포크 정보 확인
- 노드 정보 출력

9단계: 계정 백그라운드 서비스 초기화 (라인 ~1050-1100)

```
let  accounts_background_service = AccountsBackgroundService::new(
bank_forks.clone(),
exit.clone(),
AbsRequestHandlers {
snapshot_request_handler,
pruned_banks_request_handler,
},
config.accounts_db_test_hash_calculation,
last_full_snapshot_slot,
);
```

주요 작업:

- 계정 백그라운드 서비스 생성
- 스냅샷 요청 핸들러 설정
- 정리된 은행 요청 핸들러 설정

10단계: 스케줄러 풀 설정 (라인 ~1100-1150)

```
match &config.block_verification_method {
BlockVerificationMethod::BlockstoreProcessor => {
info!("no scheduler pool is installed for block verification...");
}
BlockVerificationMethod::UnifiedScheduler => {
let  scheduler_pool = DefaultSchedulerPool::new_dyn(
config.unified_scheduler_handler_threads,
config.runtime_config.log_messages_bytes_limit,
transaction_status_sender.clone(),
Some(replay_vote_sender.clone()),
prioritization_fee_cache.clone(),
);
bank_forks
.write()
.unwrap()
.install_scheduler_pool(scheduler_pool);
}
}
```

주요 작업:

- 블록 검증 방법에 따른 스케줄러 설정
- 통합 스케줄러 사용시 스케줄러 풀 설치

- 우선순위 수수료 캐시 설정

서비스 초기화 순서

의존성 기반 초기화 순서

1. **기반 시설 서비스**

- `sigverify::init()` - 서명 검증 시스템
- Rayon 스레드 풀
- 제네시스 설정 로드

2. **데이터 저장소**

- `blockstore` 로드
- `bank_forks` 초기화
- 계정 데이터베이스 설정

3. **네트워크 서비스**

- `cluster_info` 설정
- `gossip_service` 초기화
- QUIC 엔드포인트 설정

4. **처리 파이프라인**

- `poh_service` 초기화
- `tpu` (Transaction Processing Unit) 생성
- `tvu` (Transaction Validation Unit) 생성

5. **백그라운드 서비스**

- `accounts_background_service`

- `system_monitor_service`
- `stats_reporter_service`

6. **RPC 서비스**

- `json_rpc_service` (선택적)
- `pubsub_service` (선택적)

초기화 의존성 그래프

```
제네시스 설정 → 블록스토어 → 은행 포크
↓ ↓ ↓
서명검증 → 클러스터정보 → PoH 서비스
↓ ↓ ↓
네트워크 → TPU/TVU → RPC 서비스
```

오류 처리 및 복구

주요 오류 시나리오

1. **제네시스 설정 오류**

```
.map_err(|err| format!("Failed to open genesis config: {err}"))?
```

2. **블록스토어 로드 실패**

```
load_blockstore(...)?
```

3. **네트워크 설정 오류**

```
verify_net_stats_access()
.map_err(|err| format!("Failed to access network stats: {err:?}"))?
```

복구 메커니즘

- **블록스토어 백업**: 잘못된 shred 버전 감지시 자동 백업 및 정리
- **계정 경로 정리**: 시작시 자동으로 손상된 계정 데이터 정리
- **스냅샷 정리**: 불완전하거나 오래된 스냅샷 자동 제거

성능 최적화 요소

병렬 처리

1. **Rayon 스레드 풀**: CPU 집약적 작업의 병렬 처리
2. **비동기 서비스**: Tokio 런타임을 사용한 I/O 집약적 작업
3. **QUIC 프로토콜**: 고성능 네트워크 통신

메모리 관리

1. **Arc<RwLock<>>**: 스레드 안전한 공유 데이터 구조
2. **채널 기반 통신**: 무잠금 메시지 전달
3. **백그라운드 정리**: 주기적인 메모리 정리 작업

결론

Solana의 `Validator::new()` 함수는 복잡한 분산 시스템의 모든 컴포넌트를 체계적으로 초기화하는 정교한 과정입니다. 각 단계는 이전 단계의 결과에 의존하며, 오류 발생시 적절한 복구 메커니즘을 제공합니다. 이러한 설계는 Solana가 고성능과 안정성을 동시에 달성할 수 있게 하는 핵심 요소입니다.